

Informatyka w Medycynie

Raport końcowy: Automatyczna detekcja naczyń krwionośnych dna oka

Skład grupy:

Aleksander Staszewski (160143)

Krzysztof Zaporowski (160237)

1 Zastosowany język programowania i biblioteki

Projekt został zrealizowany w języku **Python 3.12**. Wykorzystano następujące biblioteki dodatkowe:

- **OpenCV (cv2)**: Podstawowa biblioteka do operacji na obrazach, filtracji i ekstrakcji momentów.
- **NumPy**: Kluczowa dla wydajnych obliczeń macierzowych i manipulacji danymi wejściowymi dla modeli.
- **scikit-image**: Wykorzystana do zaawansowanych filtrów rurkowatości (Frangi, Sato) oraz morfologii.
- **scikit-learn**: Zastosowana do implementacji klasyfikatora Random Forest oraz obliczania metryk (Acc, Sens, Spec).
- **TensorFlow / Keras**: Framework użyty do budowy i treningu głębokiej sieci neuronowej (CNN).
- **Matplotlib & IPyWidgets**: Umożliwiły stworzenie interaktywnego GUI w środowisku Jupyter Notebook.

2 Opis zastosowanych metod

2.1 Przetwarzanie obrazów

Proces przetwarzania obrazu podzielono na trzy etapy, zgodnie z wymaganiami:

i. Poszczególne kroki i zastosowane filtry:

1. **Ekstrakcja kanału zielonego**: Konwersja obrazu RGB i wybór wyłącznie składowej G.
2. **CLAHE**: Zastosowanie adaptacyjnego wyrównywania histogramu (`clipLimit=1.5`, `tileGridSize=(8,8)`).
3. **Filtracja Sato**: Wyznaczenie mapy "vesselness" na podstawie wartości własnych macierzy Hessian.
4. **Post-processing**: Filtracja medianowa (redukcja szumu) oraz usuwanie małych obiektów (`min_size=200`).

ii. Uzasadnienie rozwiązania: Wybór kanału zielonego wynika z fizjologii oka – naczynia pochłaniają najwięcej światła w tym spektrum, co daje najwyższy kontrast. CLAHE pozwala na uwydatnienie naczyń w ciemniejszych obszarach zdjęcia bez prześwietlania jasnych fragmentów (np. tarczy nerwu wzrokowego). Filtr Sato jest dedykowany do struktur rurkowatych, co stanowi doskonały punkt wyjścia (baseline) dla modeli ML.

```
1 def preprocess_image(image):
2     green_channel = image[:, :, 1]
3     clahe = cv2.createCLAHE(clipLimit=1.5, tileGridSize=(8, 8))
4     enhanced = clahe.apply(green_channel)
5     return enhanced
```

Listing 1: Implementacja przetwarzania wstępnego

2.2 Uczenie maszynowe

i. Przygotowanie danych i ekstrakcja cech: Dane przygotowujemy przez podział obrazu na wycinki o rozmiarze **5x5 px**. Dla każdego środkowego piksela wycinka obliczany jest wektor **17 cech**:

- Statystyki lokalne: średnia, wariancja, odchylenie standardowe.
- Cechy krawędzi: Sobel, Laplacian.
- Vesselness: Frangi, Sato.
- Momenty geometryczne: 10 momentów (w tym 7 niezmienników Hu).

ii. Wstępne przetwarzanie zbioru uczącego: Z uwagi na ogromną liczbę pikseli tła względem naczyń, zastosowano **undersampling**. Z każdego obrazu treningowego pobierana jest równa liczba pikseli naczyń i tła (stosunek 50:50), co zapobiega biasowi modelu w stronę klasy dominującej. Dodatkowo dane dla CNN są normalizowane (Z-score normalization).

iii. Zastosowane metody i parametry:

- **Klasyfikator ML (Random Forest):** 100 drzew decyzyjnych, brak limitu głębokości (auto).
- **Sieć Neuronowa (CNN):** Architektura wielowarstwowa.
 - 2 warstwy Conv2D (64 i 128 filtrów 3×3) z BatchNormalization.
 - Warstwa Flatten i Dense (256 neuronów) z Dropout (0.4).
 - Optymalizator: **Adam** ($lr = 5e - 4$), funkcja straty: **binary_crossentropy**.
 - Trening: 50 epok, EarlyStopping (patience=8).

```
1 model = keras.Sequential([
2     layers.Input(shape=(5, 5, 17)),
3     layers.Conv2D(64, 3, activation='relu', padding='same'),
4     layers.BatchNormalization(),
5     layers.Conv2D(128, 3, activation='relu', padding='same'),
6     layers.Flatten(),
7     layers.Dense(256, activation='relu'),
8     layers.Dropout(0.4),
9     layers.Dense(1, activation='sigmoid')
10 ])
```

Listing 2: Budowa modelu CNN (Keras)

iv. Wyniki wstępnej oceny (Hold-out): Modele oceniono na 5 obrazach testowych (ostatnie 5 z bazy), których modele nie widziały podczas treningu.

Metoda	Acc	Sens	Spec	G-Mean
Filtr Sato (Baseline)	0.9468	0.6246	0.9784	0.7815
Klasyfikator ML (RF)	0.9193	0.8505	0.9258	0.8869
Sieć neuronowa (CNN)	0.9293	0.8062	0.9413	0.8711

v. Krótkie uzasadnienie rozwiązania: Zastosowanie wektora cech (z punktu 4.0) jako kanałów wejściowych dla CNN pozwoliło sieci na naukę korelacji między zaawansowanymi deskryptorami obrazu. Dzięki temu sieć osiąga znacznie wyższą czułość (Sensitivity) niż klasyczne metody, skuteczniej wykrywając cienkie i mało widoczne naczynia.

3 Wizualizacja

W notatniku `main.ipynb` zaimplementowano funkcję wizualizacji, która dla każdego zdjęcia generuje zestawienie: Oryginał | Maska | Wyniki metod. Pod każdym zestawem generowana jest indywidualna tabela wyników, co pozwala na analizę sukcesów i porażek algorytmu (np. trudności w obszarze plamki żółtej).

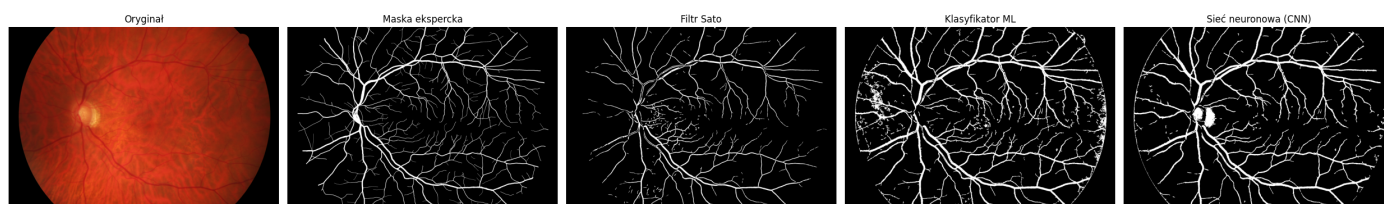
3.1 Uwagi odnośnie czasu wykonania

Nie liczę tutaj czasu treningu modeli ponieważ domyślnie dla 15 pierwszych zdjęć modele są zapisane w plikach i są tylko wczytywane. Sama segmentacja zajmuje w zależności od sprzętu od około jednej minuty (Mac mini M4), nawet do 10. Dlatego razem z rozwiązaniem zamieszczamy film ile czasu "u nas" zajmuje analiza.

4 Wizualizacja wyników

Wyniki dla 5 ostatnich zdjęć ze zbioru (zbiór trenowany na piętnastu pierwszych zdjęciach)

4.1 Zdjęcie 14g

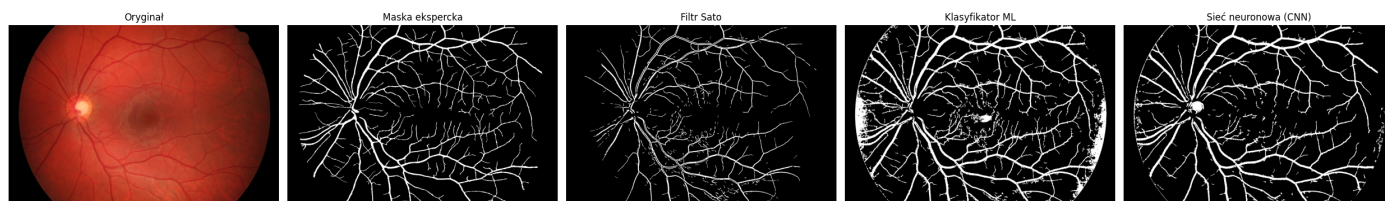


Rysunek 1: Zdjęcie 14g

Tabela 1: Porównanie wyników metod segmentacji/klasyfikacji - 14g

Metoda	Acc	Sens	Spec	G-Mean
Filtr Sato	0.9527	0.6192	0.9832	0.7802
Klasyfikator ML	0.9171	0.8421	0.9239	0.8821
Sieć neuronowa (CNN)	0.9283	0.7920	0.9407	0.8631

4.2 Zdjęcie 14h

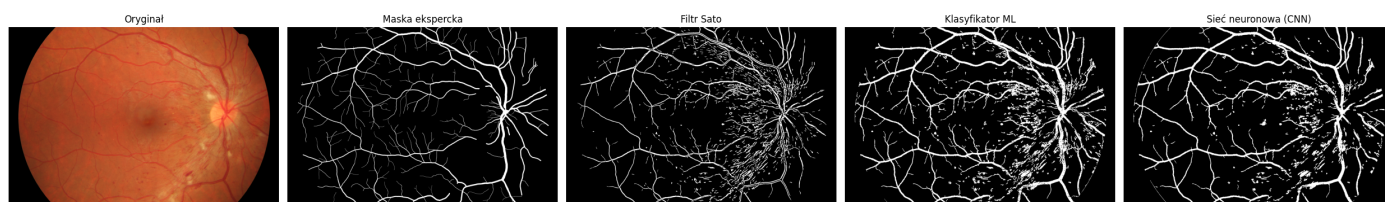


Rysunek 2: Zdjęcie 14h

Tabela 2: Porównanie wyników metod segmentacji/klasyfikacji - 14h

Metoda	Acc	Sens	Spec	G-Mean
Filtr Sato	0.9467	0.5990	0.9867	0.7688
Klasyfikator ML	0.9010	0.9078	0.9002	0.9040
Sieć neuronowa (CNN)	0.9319	0.8179	0.9451	0.8792

4.3 Zdjęcie 15dr

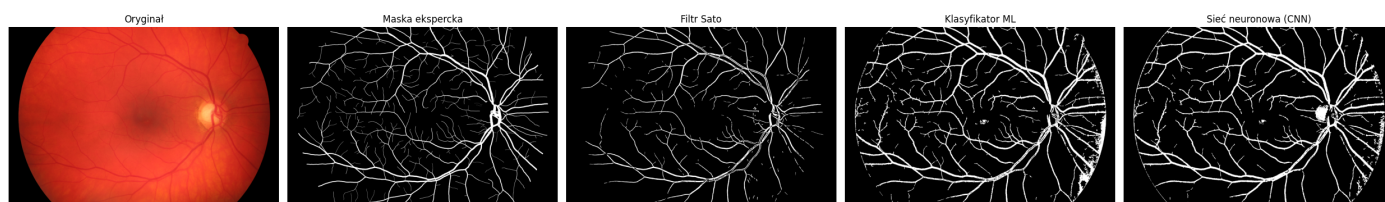


Rysunek 3: Zdjęcie 15dr

Tabela 3: Porównanie wyników metod segmentacji/klasyfikacji - 15dr

Metoda	Acc	Sens	Spec	G-Mean
Filtr Sato	0.9233	0.6546	0.9458	0.7869
Klasyfikator ML	0.9012	0.8603	0.9046	0.8822
Sieć neuronowa (CNN)	0.9172	0.8047	0.9266	0.8635

4.4 Zdjęcie 15g

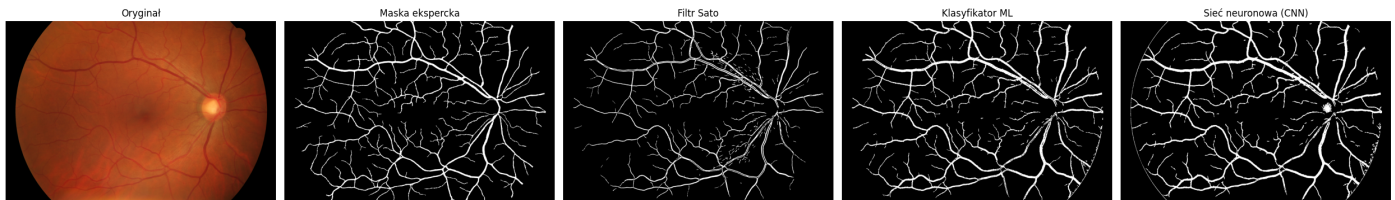


Rysunek 4: Zdjęcie 15g

Tabela 4: Porównanie wyników metod segmentacji/klasyfikacji - 15g

Metoda	Acc	Sens	Spec	G-Mean
Filtr Sato	0.9525	0.6023	0.9857	0.7705
Klasyfikator ML	0.9236	0.8267	0.9327	0.8781
Sieć neuronowa (CNN)	0.9245	0.8031	0.9360	0.8670

4.5 Zdjęcie 15h



Rysunek 5: Zdjęcie 15h

Tabela 5: Porównanie wyników metod segmentacji/klasyfikacji - 15h

Metoda	Acc	Sens	Spec	G-Mean
Filtr Sato	0.9587	0.6480	0.9905	0.8012
Klasyfikator ML	0.9535	0.8157	0.9675	0.8884
Sieć neuronowa (CNN)	0.9446	0.8134	0.9580	0.8827

5 Analiza wyników

Tabela 6: Uśrednione wyniki

Metoda	Acc	Sens	Spec	G-Mean
Filtr Sato (Baseline)	0.9468	0.6246	0.9784	0.7815
Klasyfikator ML (RF)	0.9193	0.8505	0.9258	0.8869
Sieć neuronowa (CNN)	0.9293	0.8062	0.9413	0.8711

Analizując uzyskane wyniki można zauważyć, że filtr Sato osiągał najwyższe wartości dokładności (Accuracy) oraz swoistości (Specificity). Dla wszystkich badanych obrazów dokładność tej metody przekraczała 92%, osiągając maksymalnie 95,87% dla obrazu 15h. Jednocześnie filtr Sato charakteryzował się najniższą czułością (Sensitivity), która zawierała się w przedziale od 59,90% do 65,46%. Oznacza to, że metoda skutecznie identyfikuje piksele tła, jednak pomija znaczną część naczyń krwionośnych. Potwierdzają to również najniższe wartości współczynnika G-Mean, mieszczące się w zakresie od 0,7688 do 0,8012.

Klasyfikator uczenia maszynowego uzyskał najwyższe wartości czułości dla większości obrazów. Najlepszy wynik Sensitivity wyniósł 90,78% dla obrazu 14h, natomiast najniższy 81,57% dla obrazu 15h. Wysokie wartości czułości wskazują na skuteczne wykrywanie naczyń krwionośnych. Jednocześnie swoistość tej metody była nieco niższa niż w przypadku filtra Sato, co oznacza większą liczbę fałszywie dodatnich detekcji. Pomimo tego klasyfikator ML osiągnął najwyższe wartości współczynnika G-Mean

dla wszystkich analizowanych obrazów, uzyskując wyniki od 0,8781 do 0,9040. Świadczy to o najlepszym zrównoważeniu pomiędzy czułością i swoistością. Czasami klasyfikator ML ma problemy z szumem przez co po nałożeniu wyniku na oryginalny obraz nie wygląda to zbyt estetycznie.

Sieć neuronowa CNN osiągnęła wyniki pośrednie pomiędzy filtrem Sato a klasyfikatorem ML. Dokładność klasyfikacji wynosiła od 91,72% do 94,46%, natomiast czułość od 79,20% do 81,79%. Wartości swoistości przekraczały 92% dla wszystkich obrazów. Uzyskane wartości G-Mean mieściły się w przedziale od 0,8631 do 0,8827, co wskazuje na dobrą równowagę pomiędzy wykrywaniem naczyń a poprawną identyfikacją tła. Mimo że CNN nie osiągnęła najlepszych wyników w żadnej z analizowanych metryk, jej rezultaty były stabilne dla wszystkich obrazów testowych.

Porównując wszystkie metody można stwierdzić, że filtr Sato najlepiej radzi sobie z zachowaniem wysokiej swoistości, jednak kosztem wykrywania części naczyń krwionośnych. Sieć neuronowa CNN zapewnia bardziej zrównoważone wyniki, jednak nie przewyższa klasyfikatora ML pod względem współczynnika G-Mean. Najlepsze rezultaty ogólne uzyskał klasyfikator uczenia maszynowego, który osiągnął najwyższe wartości G-Mean dla wszystkich badanych obrazów, co wskazuje na najbardziej efektywną segmentację naczyń krwionośnych w analizowanym zbiorze testowym.